

Atividade 02 — Bubble Sort: contagem de operações e comparação com o Selection Sort

ICD-004 - Análise de Algoritmos

Aula 04 - Atividade 02

Aluno: **Wellington Junior Torres de Melo**

Data: 01/09/2026

1. O que foi implementado

Foram implementados em Python o **Bubble Sort** e o **Selection Sort**, os dois instrumentados para contar **comparações** e **trocas**. O Bubble Sort compara pares vizinhos e troca quando estão fora de ordem; a cada passagem o maior elemento restante “sobe” para o fim da lista, então a passagem seguinte já olha um elemento a menos. Quando uma passagem inteira não faz nenhuma troca, a lista já está ordenada e o algoritmo para — que é o item 3 do “Como funciona” do enunciado.

Cada execução confere o resultado contra o **sorted()** antes de reportar os números, para garantir que a contagem se refere a uma ordenação de fato correta.

2. Os três cenários com $n = 5$

Cada cenário foi ordenado pelos dois algoritmos, sobre a mesma lista, contando comparações e trocas:

Cenário	Lista	Bubble comparações	Bubble trocas	Selection comparações	Selection trocas
Melhor caso (já ordenada)	[1, 2, 3, 4, 5]	4	0	10	0
Médio caso (aleatória)	[3, 5, 1, 4, 2]	10	6	10	2
Pior caso (invertida)	[5, 4, 3, 2, 1]	10	10	10	2

O contraste já aparece com $n = 5$: o Bubble Sort vai de **4 comparações** na lista já ordenada para **10** na invertida, enquanto o Selection Sort faz **10 comparações nos três casos**. Em compensação, no pior caso o Bubble Sort faz **10 trocas** contra apenas **2** do Selection Sort.

3. Crescimento das operações conforme o tamanho da lista

Para enxergar o comportamento assintótico, o mesmo teste foi repetido com listas maiores (a lista aleatória usa semente fixa, então o resultado é reprodutível):

n	Cenário	Bubble comparações	Bubble trocas	Selection comparações	Selection trocas
5	melhor	4	0	10	0
5	medio	9	4	10	2
5	pior	10	10	10	2
10	melhor	9	0	45	0
10	medio	42	12	45	6
10	pior	45	45	45	5
50	melhor	49	0	1.225	0
50	medio	1.170	533	1.225	45
50	pior	1.225	1.225	1.225	25
100	melhor	99	0	4.950	0
100	medio	4.922	2.766	4.950	96
100	pior	4.950	4.950	4.950	50
500	melhor	499	0	124.750	0
500	medio	124.372	62.535	124.750	493
500	pior	124.750	124.750	124.750	250

A tabela confirma o **$O(n^2)$** : quando n dobra de 50 para 100, as comparações do pior caso saem de **1.225** para **4.950** — cerca de **4 vezes mais**, que é exatamente o esperado de um algoritmo quadrático ($2^2 = 4$). Já o melhor caso do Bubble Sort cresce de forma linear: com n = 500 são apenas **499** comparações (n - 1) contra **124.750** no pior caso.

4. Por que o número de operações varia

O Bubble Sort só troca quando encontra um par vizinho fora de ordem, e só faz uma nova passagem se a anterior trocou alguma coisa. Ou seja: o trabalho dele depende de **quanto a lista já está ordenada**.

Melhor caso (lista ordenada): a primeira passagem não troca nada, o algoritmo percebe isso pela bandeira de controle e para. Faz apenas **n - 1 comparações** e **0 trocas** — comportamento **$O(n)$** .

Pior caso (lista invertida): todo par comparado está fora de ordem, então ele troca em *todas* as comparações e nunca consegue parar mais cedo. Faz **$n(n-1)/2$ comparações** e o mesmo número de trocas — **$O(n^2)$** .

Médio caso (lista aleatória): fica entre os dois. Em média metade dos pares está fora de ordem, então o número de trocas cai perto da metade do pior caso — na tabela, com n = 500, foram **62.535** trocas contra **124.750** do pior caso. Ainda assim é **$O(n^2)$** , porque a notação descreve o *crescimento*, não o valor exato.

5. Comparação com o Selection Sort

O Selection Sort percorre **todo** o trecho que ainda falta ordenar para achar o menor elemento, sem aproveitar a ordem que já existe. Por isso ele faz **sempre $n(n-1)/2$ comparações**, nos três cenários: com $n = 5$ são 10 comparações tanto na lista ordenada quanto na invertida. Ele é **$O(n^2)$ inclusive no melhor caso** — não tem como parar antes.

Em compensação, faz no máximo **$n - 1$ trocas**, porque cada elemento vai direto para a posição final. Com $n = 500$ no pior caso foram **250** trocas contra **124.750** do Bubble Sort — quase três ordens de grandeza de diferença em movimentação de dados.

Resumindo, quando usar cada um:

- **Lista quase ordenada** — Bubble Sort, porque ele detecta isso e para em uma única passagem, $O(n)$.
- **Trocas caras** (registros grandes, gravação em disco) — Selection Sort, porque movimenta muito menos dados.
- **Listas grandes** — nenhum dos dois: ambos são $O(n^2)$. Vale usar algoritmos $O(n \log n)$, como Merge Sort ou Quick Sort.

6. Saída completa do programa

BUBBLE SORT: CONTAGEM DE OPERACOES E COMPARACAO COM O SELECTION SORT

1) OS TRES CENARIOS COM $n = 5$

Cenario	Lista	BUBBLE SORT comp / trocas	SELECTION SORT comp / trocas
Melhor caso (ja ordenada)	[1, 2, 3, 4, 5]	4 / 0	10 / 0
Medio caso (aleatoria)	[3, 5, 1, 4, 2]	10 / 6	10 / 2
Pior caso (invertida)	[5, 4, 3, 2, 1]	10 / 10	10 / 2

2) CRESCIMENTO DAS OPERACOES CONFORME O TAMANHO DA LISTA

n	cenario	bubble comp	bubble trocas	select comp	select trocas
5	melhor	4	0	10	0
5	medio	9	4	10	2
5	pior	10	10	10	2
10	melhor	9	0	45	0
10	medio	42	12	45	6
10	pior	45	45	45	5
50	melhor	49	0	1225	0
50	medio	1170	533	1225	45
50	pior	1225	1225	1225	25
100	melhor	99	0	4950	0
100	medio	4922	2766	4950	96
100	pior	4950	4950	4950	50
500	melhor	499	0	124750	0
500	medio	124372	62535	124750	493
500	pior	124750	124750	124750	250

3) POR QUE O NUMERO DE OPERACOES VARIA?

O Bubble Sort so troca quando encontra um par vizinho fora de ordem. Logo, o trabalho dele depende de QUANTO a lista ja esta ordenada:

- Melhor caso (lista ordenada): a primeira passagem nao troca nada, o algoritmo percebe isso e para. Faz apenas $n-1 = 4$ comparacoes e 0 trocas. Comportamento $O(n)$.
- Pior caso (lista invertida): TODO par comparado esta fora de ordem, entao ele troca em todas as comparacoes. Faz $n(n-1)/2 = 10$ comparacoes e 10 trocas. Comportamento $O(n^2)$.
- Medio caso (lista aleatoria): fica entre os dois. Em media metade dos pares esta fora de ordem, entao o numero de trocas fica perto da metade do pior caso. Tambem e $O(n^2)$.

A tabela do item 2 confirma: quando n dobra, as comparacoes do pior caso ficam cerca de 4 vezes maiores - exatamente o que se espera de um algoritmo quadratico ($2^2 = 4$).

4) COMPARACAO COM O SELECTION SORT

O Selection Sort percorre todo o trecho restante para achar o menor elemento, sem aproveitar a ordem que ja existe. Por isso ele faz SEMPRE $n(n-1)/2$ comparacoes, nos tres cenarios: com $n = 5$ sao 10 comparacoes na lista ordenada, na aleatoria e na invertida. Ele e $O(n^2)$ inclusive no melhor caso.

Em compensacao, faz no maximo $n-1$ trocas, porque cada elemento vai direto para o lugar certo. O Bubble Sort chega a 10 trocas com $n = 5$.

Resumindo:

- lista quase ordenada -> Bubble Sort vence (para cedo, $O(n)$).
- trocas caras (registros grandes) -> Selection Sort vence, porque movimenta muito menos dados.
- listas grandes -> os dois sao ruins, $O(n^2)$. Vale usar algoritmos $O(n \log n)$, como Merge Sort ou Quick Sort.

7. Código-fonte

Arquivo **bubble_sort_ia.py** — é este script que ordena, conta as operações e gera o resultado.txt e o operacoes.csv usados nas seções anteriores. Os comentários foram removidos aqui para o código caber melhor na página; o arquivo original está comentado.

```
"""
ICD-004 - Analise de Algoritmos
Aula 04 - Atividade 02

Implementa o BUBBLE SORT contando COMPARACOES e TROCAS, testa os tres
cenarios (melhor, medio e pior caso) e compara com o SELECTION SORT.

Para rodar:  python bubble_sort.py
"""

import random
from pathlib import Path

PASTA = Path(__file__).parent

MELHOR_CASO = [1, 2, 3, 4, 5]
MEDIO_CASO = [3, 5, 1, 4, 2]
PIOR_CASO = [5, 4, 3, 2, 1]

TAMANHOS = [5, 10, 50, 100, 500]
SEMENTE = 42

def bubble_sort(lista_original):
    lista = list(lista_original)
    comparacoes = 0
    trocas = 0
    n = len(lista)

    for passagem in range(n - 1):
        houve_troca = False

        for i in range(n - 1 - passagem):
            comparacoes += 1
            if lista[i] > lista[i + 1]:
                lista[i], lista[i + 1] = lista[i + 1], lista[i]
                trocas += 1
                houve_troca = True

        if not houve_troca:
            break

    return lista, comparacoes, trocas

def selection_sort(lista_original):
    lista = list(lista_original)
    comparacoes = 0
    trocas = 0
    n = len(lista)

    for inicio in range(n - 1):
        menor = inicio

        for i in range(inicio + 1, n):
            comparacoes += 1
            if lista[i] < lista[menor]:
                menor = i

        if menor != inicio:
            lista[inicio], lista[menor] = lista[menor], lista[inicio]
            trocas += 1

    return lista, comparacoes, trocas

relatorio = []

def mostrar(texto=""):
    print(texto)
    relatorio.append(texto)

mostrar("BUBBLE SORT: CONTAGEM DE OPERACOES E COMPARACAO COM O SELECTION SORT")
mostrar("=" * 70)
mostrar()
```

```

cenarios = [
    ("Melhor caso (ja ordenada)", MELHOR_CASO),
    ("Medio caso (aleatoria)", MEDIO_CASO),
    ("Pior caso (invertida)", PIOR_CASO),
]

mostrar("1) OS TRES CENARIOS COM n = 5")
mostrar("-" * 70)
mostrar(f" {'Cenario':<26} {'Lista':<17} "
        f"{'BUBBLE SORT':<16} {'SELECTION SORT':<16}")
mostrar(f" {'':<26} {'':<17} "
        f"{'comp / trocas':<16} {'comp / trocas':<16}")

linhas_cenarios = ["cenario,lista,bubble_comparacoes,bubble_trocas,"
                  "selection_comparacoes,selection_trocas"]

for nome, lista in cenarios:
    saida_bubble, comp_b, troc_b = bubble_sort(lista)
    saida_selection, comp_s, troc_s = selection_sort(lista)

    assert saida_bubble == saida_selection == sorted(lista)

    mostrar(f" {nome:<26} {str(lista):<17} "
            f"{'comp_b:>4} / {'troc_b:<9} {'comp_s:>4} / {'troc_s:<9}")
    linhas_cenarios.append(f'{nome},{lista},{comp_b},{troc_b},'
                           f'{comp_s},{troc_s}')
mostrar()

mostrar("2) CRESCIMENTO DAS OPERACOES CONFORME O TAMANHO DA LISTA")
mostrar("-" * 70)
mostrar(f" {'n':>4} {'cenario':<8} "
        f"{'bubble comp':>12} {'bubble trocas':>14} "
        f"{'select comp':>12} {'select trocas':>14}")

random.seed(SEMENTE)
linhas_csv = ["n,cenario,bubble_comparacoes,bubble_trocas,"
              "selection_comparacoes,selection_trocas"]

for n in TAMANHOS:
    listas_do_tamanho = [
        ("melhor", list(range(1, n + 1))),
        ("medio", random.sample(range(1, n + 1), n)),
        ("pior", list(range(n, 0, -1))),
    ]

    for rotulo, lista in listas_do_tamanho:
        _, comp_b, troc_b = bubble_sort(lista)
        _, comp_s, troc_s = selection_sort(lista)
        mostrar(f" {n:>4} {rotulo:<8} "
                f"{'comp_b:>12} {'troc_b:>14} {'comp_s:>12} {'troc_s:>14}")
        linhas_csv.append(f'{n},{rotulo},{comp_b},{troc_b},{comp_s},{troc_s}')
    mostrar()

tamanho = len(MELHOR_CASO)
maximo = tamanho * (tamanho - 1) // 2

mostrar("3) POR QUE O NUMERO DE OPERACOES VARIA?")
mostrar("-" * 70)
mostrar(" O Bubble Sort so troca quando encontra um par vizinho fora de")
mostrar(" ordem. Logo, o trabalho dele depende de QUANTO a lista ja esta")
mostrar(" ordenada:")
mostrar()
mostrar(" - Melhor caso (lista ordenada): a primeira passagem nao troca")
mostrar(" nada, o algoritmo percebe isso e para. Faz apenas n-1 =")
mostrar(f" {tamanho - 1} comparacoes e 0 trocas. Comportamento O(n).")
mostrar()
mostrar(" - Pior caso (lista invertida): TODO par comparado esta fora de")
mostrar(" ordem, entao ele troca em todas as comparacoes. Faz")
mostrar(f" n(n-1)/2 = {maximo} comparacoes e {maximo} trocas. Comportamento O(n^2).")
mostrar()
mostrar(" - Medio caso (lista aleatoria): fica entre os dois. Em media")
mostrar(" metade dos pares esta fora de ordem, entao o numero de trocas")
mostrar(" fica perto da metade do pior caso. Tambem e O(n^2).")
mostrar()
mostrar(" A tabela do item 2 confirma: quando n dobra, as comparacoes do")
mostrar(" pior caso ficam cerca de 4 vezes maiores - exatamente o que se")
mostrar(" espera de um algoritmo quadratico (2^2 = 4).")
mostrar()

mostrar("4) COMPARACAO COM O SELECTION SORT")
mostrar("-" * 70)

```

```

mostrar(" O Selection Sort percorre todo o trecho restante para achar o")
mostrar(" menor elemento, sem aproveitar a ordem que ja existe. Por isso")
mostrar(" ele faz SEMPRE n(n-1)/2 comparacoes, nos tres cenarios: com")
mostrar(f" n = 5 sao {maximo} comparacoes na lista ordenada, na aleatoria e na")
mostrar(" invertida. Ele e O(n^2) inclusive no melhor caso.")
mostrar()
mostrar(" Em compensacao, faz no maximo n-1 trocas, porque cada elemento")
mostrar(f" vai direto para o lugar certo. O Bubble Sort chega a {maximo} trocas")
mostrar(f" com n = {tamanho}.")
mostrar()
mostrar(" Resumindo:")
mostrar(" - lista quase ordenada -> Bubble Sort vence (para cedo, O(n)).")
mostrar(" - trocas caras (registros grandes) -> Selection Sort vence,")
mostrar(" porque movimenta muito menos dados.")
mostrar(" - listas grandes -> os dois sao ruins, O(n^2). Vale usar")
mostrar(" algoritmos O(n log n), como Merge Sort ou Quick Sort.")

(PASTA / "resultado.txt").write_text("\n".join(relatorio), encoding="utf-8")
(PASTA / "operacoes.csv").write_text("\n".join(linhas_csv), encoding="utf-8")
(PASTA / "cenarios.csv").write_text("\n".join(linhas_cenarios), encoding="utf-8")

print()
print("Resultados salvos em: resultado.txt, operacoes.csv e cenarios.csv")

```